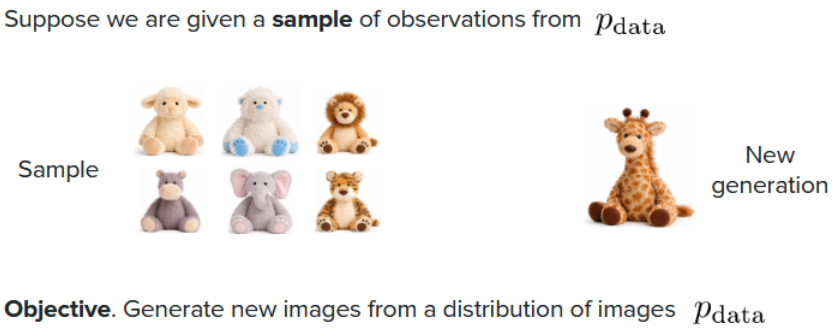
Diffusion

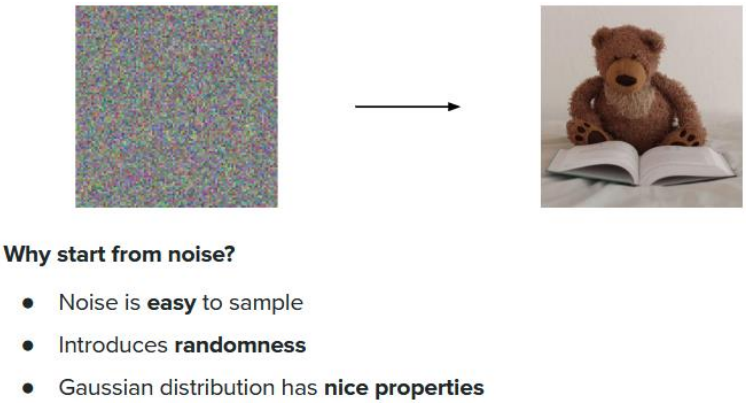
Introduction

We'll assume that there is nothing as an as input (unconditioned generations). We have some distribution of data that we use as training sets and what we want is to generate images that are like them. In mathematical terms we want to have a model that is able to **sample an observation** from that data distribution.

Let's suppose we have a **set of images**: a collection of teddy bears as **training images**. Our goal is to understand a way to **sample an observation** that is new from data distribution. We want to be able to generate an image that is in training data distribution (for instance giraffe).



Now we think "generate new images" as unconditioned generation. We just want to generate a sample that is following that distribution.

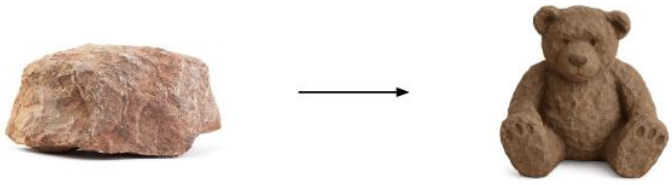


We'll see that most of all methods out there, they all have something in common: they all start their image generation process from **noise** and then they **refine** the image to be the final image.

We choose to start from noise for the following motivations:

- it is **easy to sample** from noise. It's actually something that you can sample from an easy distribution, the **gaussian distribution**.
- Noise has some **randomness** that allows you to generate different images. So even if let's say you have a condition, if you start from noise that is different from another generation, you will be able to end up with a different end image just because the starting point is different.
- Noise is basically equal to a gaussian distribution and that distribution just have very **nice properties**. So, it simplifies a lot of the math.

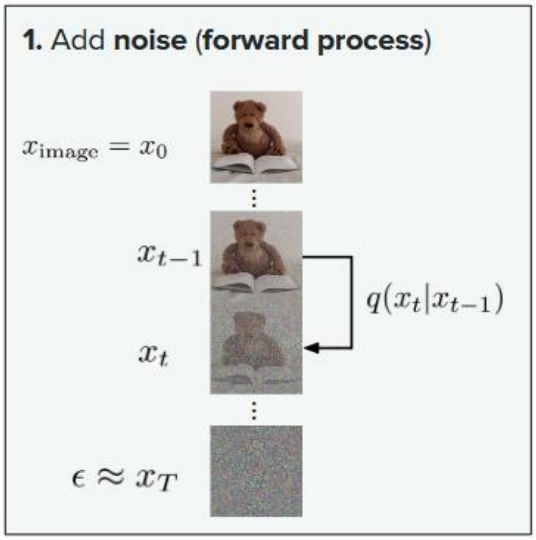
One analogy about image generation process is like if you were a sculpture and you wanted to, you know, sculpture from rock.

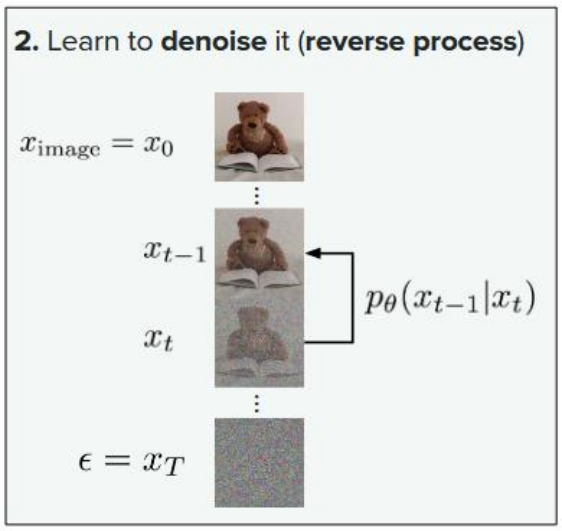


> *The sculpture is already complete within the marble block, before I start my work. It is already there, I just have to chisel away the superfluous material.*
>
> –Michelangelo

Intuition behind diffusion

DDPM (paper) that was the first at applying this *diffusion paradigm* for images in a successful way. The idea that we have is to generate new images.





We have a set of training images and our goal is "if we were to start from noise how would we be able to recover those images" (that's what we want to learn) because as we mentioned before our strategy is to start from noise and end up with a clean image.

We want to use those training set images to learn that denoising process.

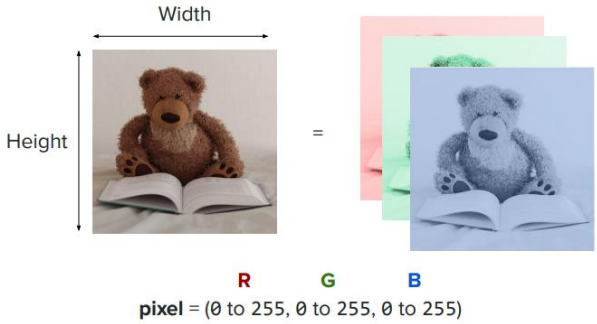
We have two processes:

- **Forward process**: take an image from the training set and we are actually corrupting that image. We're adding some noise. So here we have some process that we define (we know what $q(x_t|x_{t-1})$ is in the forward process) that goes from some image x_{t-1} and then produces an image x_t by adding some noise.
- **Reverse process**: start from the noise distribution (easy to sample) and then go through the reverse process to actually denoise that sample back to the clean space. What you want to **learn is a model** (**p**) that we'll characterize with the parameters theta ($\mathbf{p_\theta}$). What you want to learn is how you would go from a noisy image to a less noisy one. We don't know what is x_t.

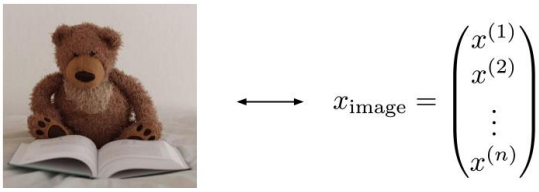
Image representation

As you know an image is composed of pixels: a height and width. Each pixel is actually composed of three values (red, green and blue) and this allows you to cover a number of colours. Moreover, you can represent your image as a function of all the pixels that are composing that image where each of these pixels contains some value for each of these colours.

So, you can think of each pixel as having three values: one that represents the amount of red, the other one that represents the amount of green and the other one that represents the amount of blue.



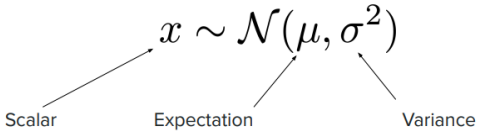
So based on that it is fair to say that you can represent images in a **vectorial fashion**. So here the dimensionality of the image would be equal to *number of pixels* times the number of dimensions you need to *represent a pixel* (which is three). So here you would have *n* dimensions where *n* is equal to number of pixels time three.



So, this would be one way to represent your images. Of course, this is not the most clever way of representing images and we will see later. When we say x_0, x_t and so on, what we're talking about is actually vectors. These vectors are the ones that are representing these images.

Probability

You may know the common distribution that's called the Gaussian distribution or normal distribution where you have some random variable that is drawn from the distribution of mean (μ) and then variance sigma squared (σ^2).



μ (mu) mean: where the peak sits.

σ^2 (sigma squared) variance: how wide the bell is. Variance is a measure of dispersion, meaning it is a measure of how far a set of numbers are spread out from their average value.

Well, you can have the same thing for vectors:

$$x \sim \mathcal{N}(\mu, \Sigma)$$

Vector: $\begin{pmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n)} \end{pmatrix}$ Expectation: $\begin{pmatrix} \mu^{(1)} \\ \mu^{(2)} \\ \vdots \\ \mu^{(n)} \end{pmatrix}$ Covariance: $\begin{pmatrix} \Sigma^{(1,1)} & \Sigma^{(1,2)} & \dots & \Sigma^{(1,n)} \\ \Sigma^{(2,1)} & \Sigma^{(2,2)} & \dots & \Sigma^{(2,n)} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma^{(n,1)} & \Sigma^{(n,2)} & \dots & \Sigma^{(n,n)} \end{pmatrix}$

We can also have random variables that are vectors and in this case you will have a vector that is drawn from a distribution of some mean (it's a vector) and then some variance which is actually represented by a covariance matrix.
Most of the time we actually simplify the covariance matrix to be a variance times the identity and we call this formulation isotropic gaussian because what that means is that the **variation of the data across all directions is the same**.
Most of the time to just simplify the math what we will assume is that the covariance is actually equal to $\sigma^2 I$ which is the diagonal matrix and not the super complicated covariance matrix.

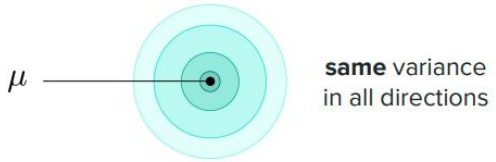
$$x \sim \mathcal{N}(\mu, \Sigma)$$

$$x^{(i)} \sim \mathcal{N}(\mu^{(i)}, \Sigma^{(i,i)}) \quad \text{and} \quad \mathrm{Cov}(x^{(i)}, x^{(j)}) = \Sigma^{(i,j)}$$

Good news. We will mainly (if not only) be dealing with **isotropic** Gaussians:

$$\Sigma = \sigma^2 I = \begin{pmatrix} \sigma^2 & 0 & \cdots & 0 \\ 0 & \sigma^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma^2 \end{pmatrix}$$

The covariance is a quantity that quantifies how two random variables vary in a linear way (general intuition).
Speaking of intuition when we say that a distribution is isotropic this is what we mean:



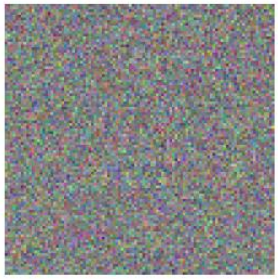
We mean that the variance is the same across all directions. So, you can think of it as some circle where the colour represents the density.

What's great about Gaussian is you also have an analytical expression that's quite simple about the probability density function.

$$p(x) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{\|x-\mu\|^2}{2\sigma^2}\right)$$

probability density
is known and "easy"

Going back to what we call noise. We saw that images can be represented by vectors. When we say we sample from noise, we will say here that we actually sample from a standard normal distribution.



$$\longleftrightarrow \quad \epsilon = \begin{pmatrix} \epsilon^{(1)} \\ \epsilon^{(2)} \\ \vdots \\ \epsilon^{(n)} \end{pmatrix} \sim \mathcal{N}(0, I)$$

In other words, $\epsilon^{(i)} \sim \mathcal{N}(0,1)$ independent and identically distributed

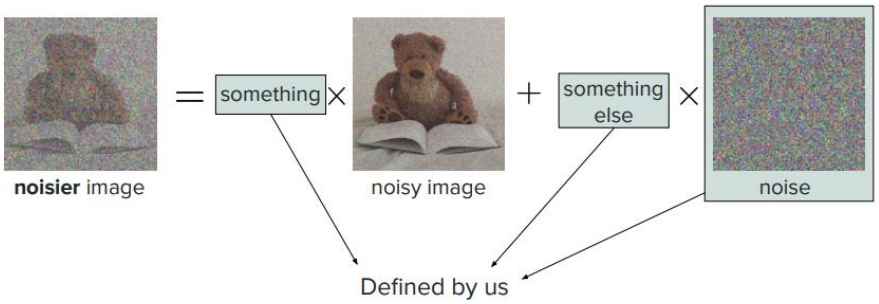
Epsilon will be drawn from a normal distribution of mean zero. So 0 is actually the zero vector and then the covariance will actually be the identity matrix.

Is the representation continuous or discrete? We talked about pixel values that were discrete 0, 1, 2 until 255. But in practice, these are actually scaled between let's say minus one and one or like some value that's more of a float. You can assume that these are actually continuous.

Forward process

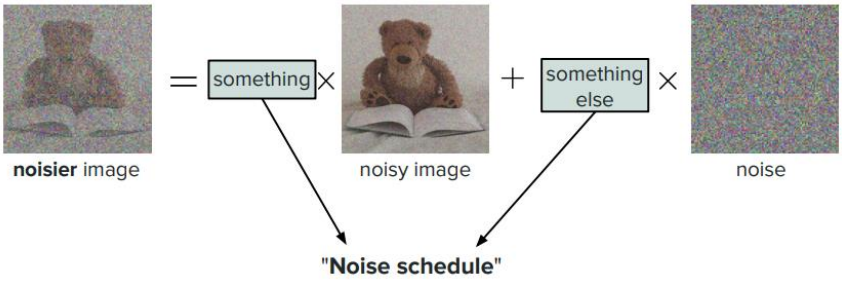
As I mentioned what we want to do is to **take clean images and to corrupt them**. We take some image that we have noised, we add some noise and we actually add these quantities in a weighted fashion and these weights are actually defined by us.

q is **chosen** by us and its **distribution** is **known** (reminder: it is stochastic!)

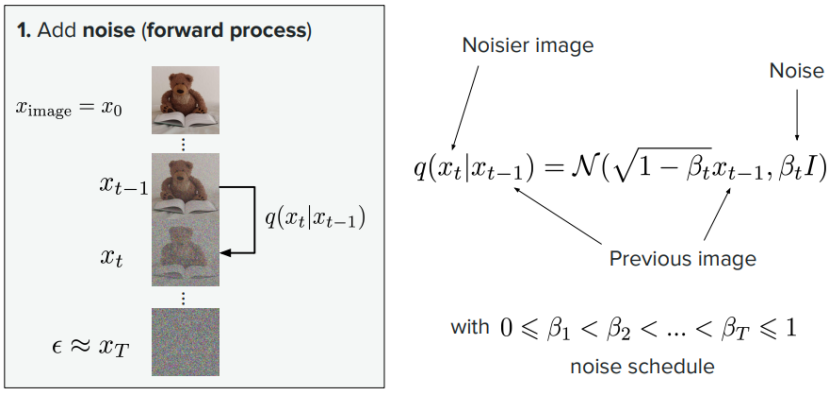


If you remember when we talked about **q** (the process that we define for the forward process) is actually something that we define. So this is how we noise images. We take an image, we take some noise from the normal distribution and then we do a weighted average with some weights.

You will hear of a term recurring term across all papers, **noise schedule**: is referring to the coefficients here that you use to multiply these values. It's a schedule because at some point you may add maybe less noise than some other points.



We start from x_0 and then the way we adding noise is we have our forward process that we've defined q which is actually something we will draw from a normal distribution. So we can express the quantity q of x_t (so the noisier image) compared to the less noisy image as a normal distribution centred on that less noisy image with some coefficient where you add some noise which is something that is represented by the covariance matrix $\beta_t I$.



$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\epsilon_t \text{ where } \epsilon_t \sim N(0; I)$$

You taking your less noisy image (x_{t-1}), waiting that by some coefficient $\left(\sqrt{1-\beta_t}\right)$ and then adding some noise ϵ with some coefficient $\left(\sqrt{\beta_t}\right)$.

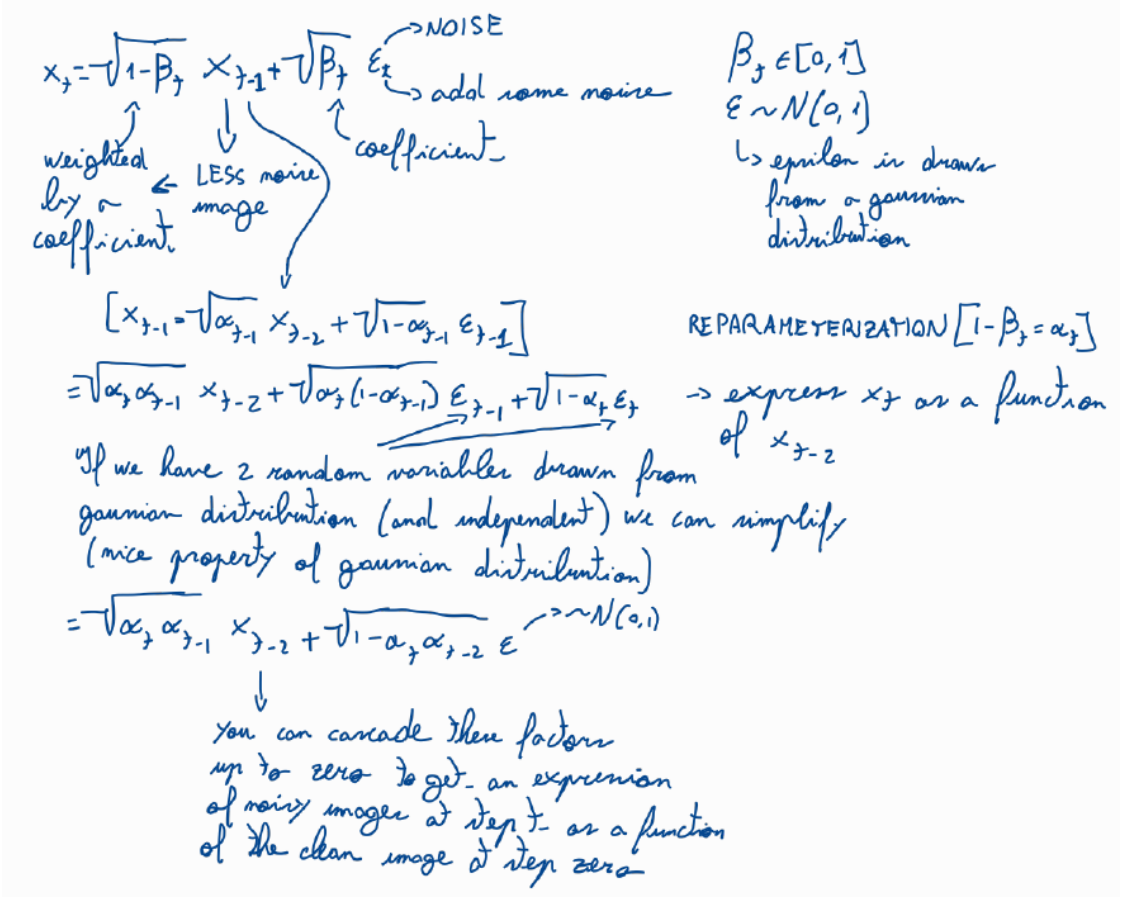
You can see that the variance of this whole thing is preserved from one step to another. So you will see that this way of noising images is called **variance preserving** (VP).

The noise schedule, so these betas are not typically uniform. They're varying and the way they are defined in the DDPM paper is they're **gradually increased** as you will start from your clean image and you will just add a little bit of noise and then maybe a little bit more and again and again and then at the end you will have much bigger noise. And the reason why people do that is because when you're close to the clean image, what you care about is to learn those **little details**. But what you're when you're closer to the noise, what you care more about learning is the **general shape** of the image

So this noise schedule is actually quite important in our **ability to learn** and it's just one way of doing things. As you as you can tell it's between zero and one. Well, because otherwise you would have a negative value in the square root.

If you define your process like this, then it turns out you don't need to noise your images in a sequential way. You can actually go directly from your clean image to the step number *t*.

Short proof:



$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

$$\downarrow$$

$$\boxed{q(x_t|x_0) = \mathcal{N}(\sqrt{\overline{\alpha}_t}x_0, (1-\overline{\alpha}_t)I)}$$

with $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$ and $\alpha_t = 1 - \beta_t$

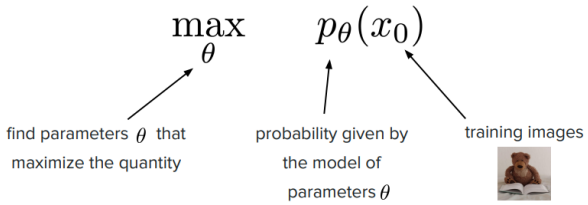
Derivation: Variance of sum of independent Gaussians is sum of respective variances.

The gaussian had very nice properties. You can have an expression of the noisy image at step t as a function of the clean image at step zero. This can be expressed as a function of $\overline{\alpha}$ where it's equal to the product of the α_i. You can express x_t as a function of x_0 by just using $\overline{\alpha}_t$ instead of β_t.

We have just defined our forward process and we just also derived a convenient way to noise our image from step zero to step t without going through all the all the steps.

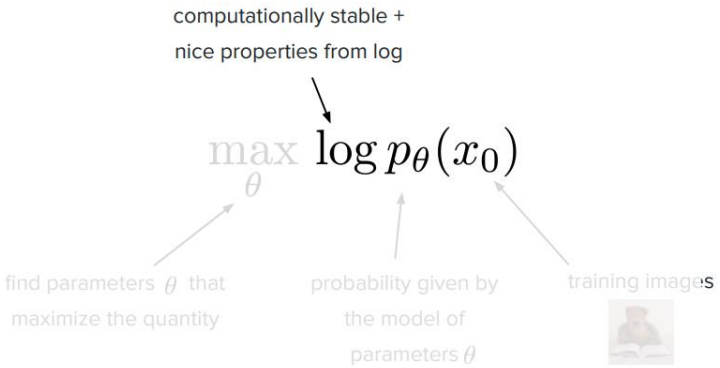
Reverse process

What we will do now is to see how we can actually determine the *process that denoises* that forward process. How can we see what $p_\theta(x_t \mid x_{t-1})$ is (we want to learn p_θ).
The common trick to do is (given some training data) we want to do is to maximize the probability of your model seeing the training data maximized (we want to maximize such a probability with respect to theta).



We want to select that parameters (ϑ) that make the observed data the most likely.

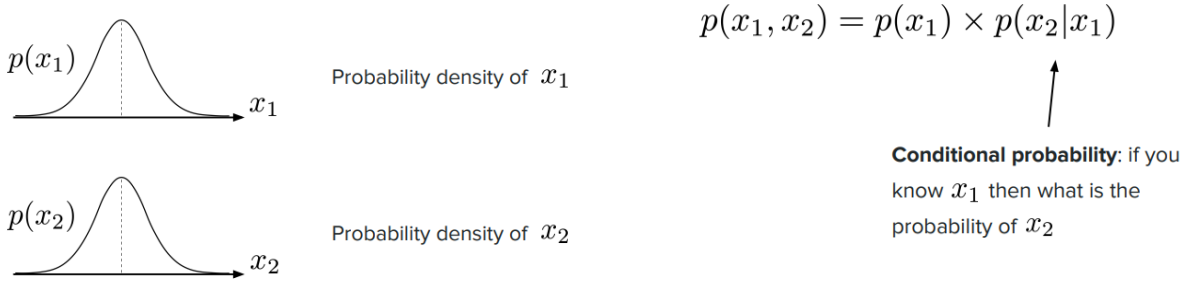
We want to find the parameters theta that maximize the probability of your model of seeing your training data. In this case training data is just our training images (it's clean images) and then our model is something that has p_θ in it.



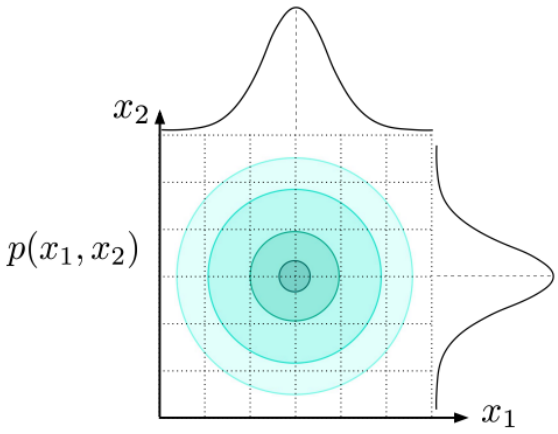
The logarithm function has pretty nice properties and it's just computationally more stable because probabilities are between zero and one. So if you have very small values of p then it can be a little bit unstable.

Joint probability distribution

If you suppose that you have two random variables that are drawn from two distributions (let's say it's x_1 and x_2, the random variables) then you can compute the joint probability distribution by taking the probability density of x_1 and multiplying that by the conditional probability of x_2 given x_1.

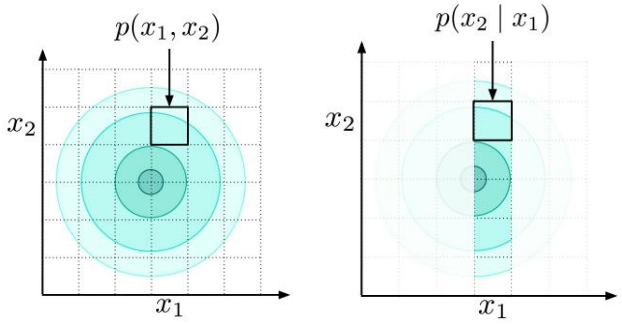


This is just by definition it's the probability chain of rule. In practice when you have a joint probability distribution one way you can represent that is through a heat map.



Joint probability $p(x_1, x_2)$ is the probability that we have x_1 **and** x_2

You have your two axis, one is representing the one along x_1 and the other one along x_2 and then the colour that you see here is the density value of this joint probability distribution.



Conditional probability $p(x_2 \mid x_1)$ is the probability that we have x_2 **knowing that** we have x_1

So this is for instance you know one square and the way you would go about doing this for the conditional probability is you would actually just look at one column and the probability of x_2 given x_1 would be the proportion of the value of the joint probability distribution in that square over the sum of all the squares for that given value x_1.

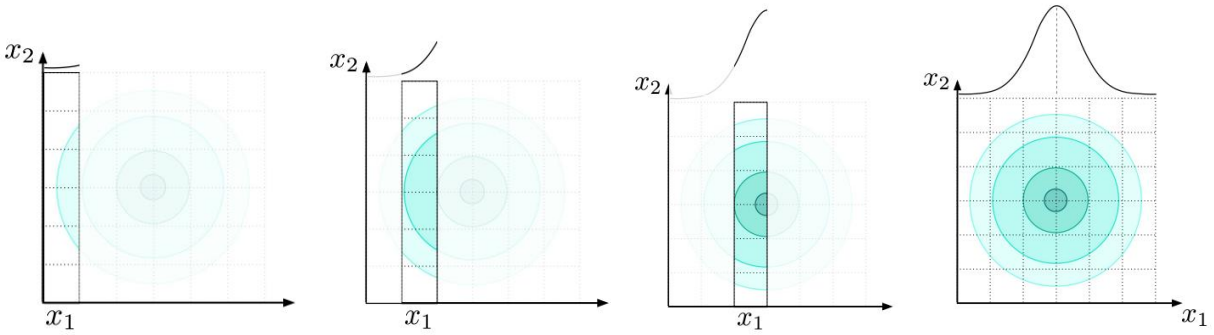
Marginalization

We call marginalization which is if you want to obtain the probability of just one random variable then what you can do is to sum the joint probability of that random variable and a second random variable and you can sum it along that second variable.

$$p(x_1) = \int p(x_1, x_2) dx_2$$

Sum across all x_2

In the following figure, so the way you would obtain $p(x_1)$ is you sum all the squares relative to x_1 again and again and you can reconstitute your probability distribution.



So that's the way you would see this from a graphical perspective. We saw that when we only have two random variables, but you can very well do that when we have t random variables.

Joint probability distribution.

$$p(x_1, x_2, ..., x_T) = p(x_1) \times p(x_2|x_1) \times ... \times p(x_T|x_1, ..., x_{T-1})$$

Marginalization.

$$p(x_1) = \int p(x_1, x_2, ..., x_T) dx_2 ... dx_T$$

We have the probability chain rule. So you have $p(x_1)$ x $p(x_2)$ given x_1 etc times $p(x_t)$ given x_1 up until x_{t-1}. So this is just extending that to a dimension of t. And one notation that you will see very often in papers is the following one. The joint probability distribution of t variables is usually noted $p(x_1)$ t.

$$p(x_1, x_2, ..., x_T) = p(x_{1:T})$$

And with that simplified notation you can just rewrite the formulas from before as follows.

Joint probability distribution (Simplified notation).

$$p(x_{1:T}) = p(x_1) \prod_{t=2}^{T} p(x_t|x_{1:t-1})$$

Marginalization (Simplified notation).

$$p(x_1) = \int p(x_{1:T}) dx_{2:T}$$

So now if we were to come back to the problem that we had around finding parameters data that maximize the probability of our model seeing the training data here what we want to do is to express $p_\theta(x_0)$ or $\log p_\theta(x_0)$.

We've seen a few paragraphs ago that the way we obtain x_0 is sampling from a noise distribution at time t and then going step by step from t to t-1 etc up until zero. To compute the quantity log $p_\theta(x_0)$, one idea for you could be to take the joint probability distribution over all the time steps and then sum across all these different quantities except x_0 aka you can just marginalize.

One idea is to say that the log p_θ (x_0) is the logarithm of the integral of p theta of x0 and then x1 up to t d of x1 to t. This is just the definition of marginalizing.

$$\log p_\theta(x_0) = \log \int p_\theta(x_0, x_{1:T})\, dx_{1:T}$$

The problem with this formulation is that it's something we cannot compute because here we're saying (let's marginalize) let's sum across all possible trajectories from noise to clean image. The problem is there are like many of them and these x's are vectors and they can be pretty big.
So long story short is this is in practice something that we cannot compute which is the reason why we have a recipe that we will go through in the next paragraph as to how we can compute that using the forward process that we have defined.

Derivation of a tractable loss
The whole strategy can be sum up as follows:

1) Derive lower bound for maximum (log-)likelihood estimation
2) Expand terms of lower bound
3) Show lower bound is tractable
4) Deduce final loss function

1° step: derive lower bound for maximum (log-)likelihood estimation
Let's start with the first step. So, I told you that the previous expression is something that is actually not something we can compute because it's just intractable. So, a common trick that people use is to find something that is easy to compute that is not necessarily the quantity of interest but maybe a **bound that is close** to what you want.

$$\mathbb{E}_{x_0 \sim q(x_0)}\left[\log p_\theta(x_0)\right] \geqslant \mathbb{E}_{x_{0:T} \sim q(x_{0:T})}\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$$

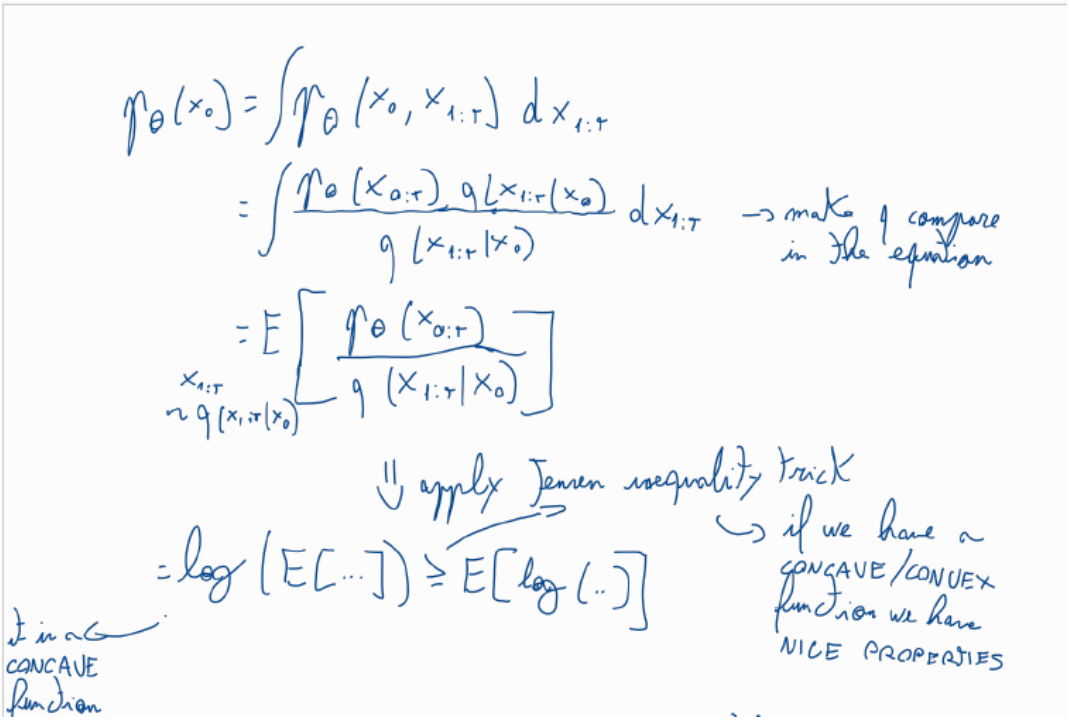
ELBO = **E**vidence **L**ower **BO**und

Derivation: Use Jensen's inequality on a convenient variational distribution

So here what you want is to maximize the expression log p_θ (x_0). If you were to maximize something that you know is lower than that (a lower bound) then it would also allow you to gain some confidence that you're maximizing the right thing.
ELBO stands for Evidence Lower Bounds and it's an expression of a quantity that is a lower bound of the quantity that we care about.

The trick is that we have our forward process that we have defined (q). We want to involve q in some way because we know what q is. q is forward process that we have defined you know when we do the noise schedule.

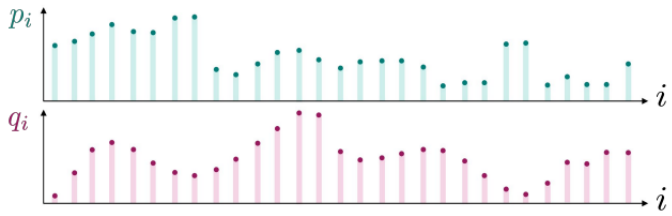
What we want to do is to express that as a function of q:



One way we progressed, is instead of summing through all possible trajectories now we're actually sampling from trajectories that were drawing from the forward process (which we know). So you may see this formula and wonder okay it seems super complicated like “how can I compute that?” right now you do not know that it is tractable I'm just telling you but the intuition here is we're actually not summing through all possible trajectories we're actually summing through trajectories that were sampling from the forward process

2° step: expand terms of the lower bound
Step number two is proving or showing that this is something that you can compute.
KL divergence: if you take two probability distributions P and Q and you want to measure how far apart they are, then there is some operator like KL divergence which quantifies how far apart they are.

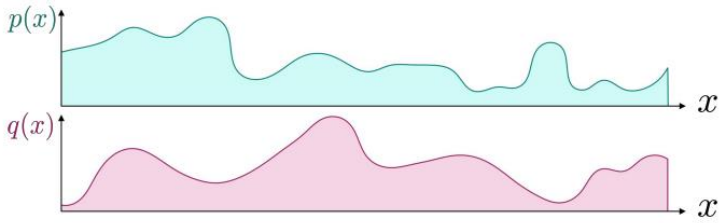
$$\mathrm{KL}(P\|Q) = \sum_{i=1}^{n} p_i \log\left(\frac{p_i}{q_i}\right)$$



KL divergence (discrete)

By taking the sum of the log of the ratio between these two probabilities weighted by the probability distribution p. We're going to be mostly in the continuous world.

$$\mathrm{KL}(P\|Q) = \int \log\left(\frac{p(x)}{q(x)}\right) p(x)\, dx = \mathbb{E}_{x\sim p}\left[\log\left(\frac{p(x)}{q(x)}\right)\right]$$



KL divergence (continuous)

And in the continuous world the KL divergence is this but in integral form. And it's the same as saying you take the expectation of the logarithm of the ratio. If you were to assume that x is sampled through p.

$$\mathrm{ELBO} = -\sum_{t=2}^{T} \mathrm{KL}\big(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\big) + \text{some other terms}$$

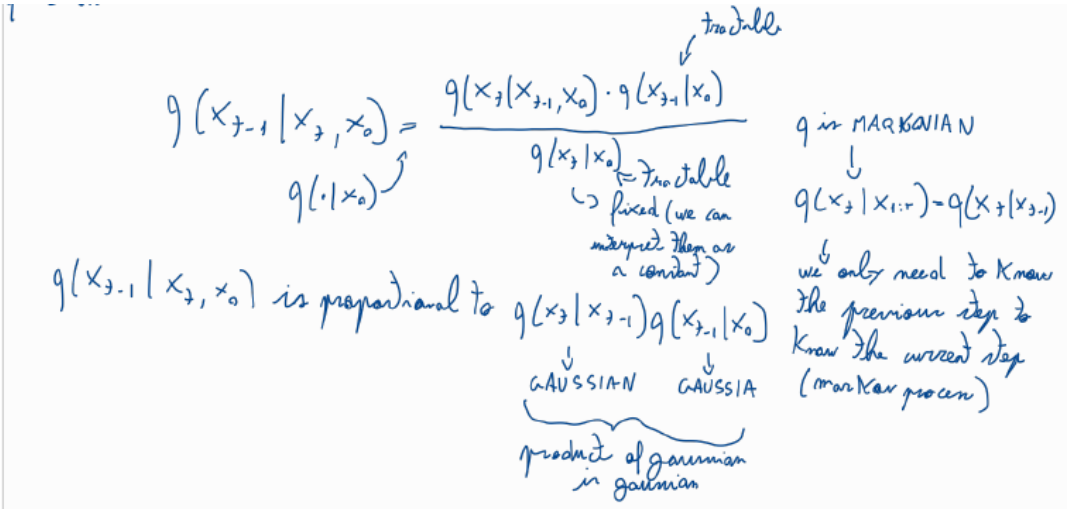
Can we compute this? — We want to learn θ

Derivation: Use properties of the log function and rearrange terms

So given that we can prove that the ELBO term that we the lower bounds it can actually be expressed with terms that we actually don't really care and we're just going to ignore plus a term which is the kl divergence of $q(x_{t-1} \mid x_t, x_0)$ (that's the first distribution). The second distribution is actually the probability of having the less noisy image x_{t-1} given x_t which is the thing that we want to learn.
The reason why we're happy is because we have a clear comparison between two probability distributions. One that we care about and another one that is potentially something that is tractable. And so just to tell you what $q(x_{t-1} \mid x_t, x_0)$ means: if I were to give you a noisy image and a clean image, my question for you would be what would be the less noisy image if I were to give you this two? So the good thing is that at training time you know x_0 so you can compute this quantity.

3° step: show lower bound is tractable
So a natural step for us is now to show that this quantity that I mentioned is indeed tractable. So we're going to see what $q(x_{t-1} \mid x_t, x_0)$ is. So again it is the fact of wondering what the distribution of the less noisy image is if I were to not only give you the more noisy image but also the clean image (which is actually a lot of information).

So you may remember or you may know may have heard of Bayes rule. Knowing that it's actually quite easy to show that this this thing is tractable.



I'm not going to derive everything but trust me that the form of $q(x_{t-1} \mid x_t, x_0)$ is a normal distribution in particular with the mean of this formula:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t)$$

sampled noise used to go from clean to noisy image

with $\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_t \right)$

noisy image

Derivation: Compute density with Bayes' rule via $q(. \mid x_0)$ + Markov property

It's a function of the noisy image and then the sampled noise that allows you to go from X_0 to X_T. This distribution is exact meaning we have not approximated anything, and this is something that we know because we have defined our forward process. Here we're cheating a little bit because we're saying what is the less noisy distribution if I give you the noisier one but also the clean image. It's a little bit of cheating because at training time we know what x_0 is. Of course, at inference time we do not.

So we know one term. The second term is this $p_\theta (x_{t-1} \mid x_t)$

b Show $p_\theta(x_{t-1} \mid x_t)$ is tractable

$$\text{ELBO} = -\sum_{t=2}^{T} \text{KL}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t)) + \text{some other terms}$$

We assume that p_θ is Gaussian:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)\big)$$

Justification: If $q(x_t|x_{t-1})$ Gaussian, then $q(x_{t-1}|x_t)$ is approx. Gaussian

Now let's show that it is tractable. It's going to be easy because we can choose what p_θ is. And turns out that it's very fair for us to assume that it is also a gaussian just because the forward process of x_{t-1} give x_t is also approximately a Gaussian. But all of that to say that in this part we're just making the assumption that what we want to learn is a gaussian. What we end up with is a kl divergence of a gaussian with a gaussian. We know the probability density function of a gaussian.

4° step: deduce loss function

So DDPM is this whole framework of defining a forward process in order to determine how you want to reverse the process.

Loss of DDPM:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,x_0,\epsilon}\left[\left\|\epsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\, t\right) - \epsilon\right\|^2\right]$$

$t \sim \mathcal{U}\{1, T\}$ $\quad x_0 \sim q_0(x_0)$ $\quad \epsilon \sim \mathcal{N}(0, I)$

Derivation: Compute KL divergence between two Gaussians

If you replace these into the KL divergence formula which is this integral of log of P over Q and then P. Then turns out you have a lot of things that simplify and at the end of the day what you end up with is just the distance between the noise that you want to predict and the noise that you sample to go from the clean image to the noise image. take the L2 distance of that.

At the end of the day, we end up with a loss that is an L2 regression on the noise that we have added to our clean image to obtain the noisy image. You just have to regress your noise from the clean image to the noisy image as a training loss